Lab 2: Fine-tune vs prompt

Does fine-tuning beat your best prompt?

Aarne Talman

5 November 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

The question

Lab 2

For *your* task, does fine-tuning beat your best prompt?

You compare two ways of getting a model to do your task:

- **prompting**: give the model instructions and, optionally, a few examples, without changing it
- **fine-tuning**: train the model on your labelled examples

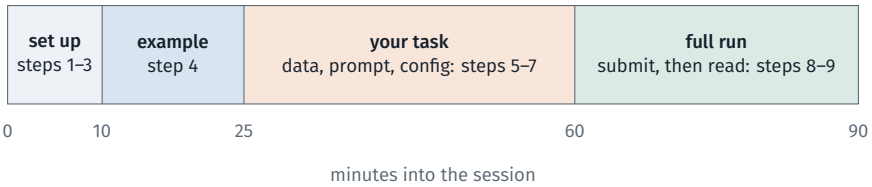
Both are scored on the same test examples, which were not used for anything else.

For the final report

- one results table (`summarize_results.py` prints it)
- one loss plot
- one paragraph: was fine-tuning worth the extra time and GPU use?

Everything you need is in `Week-2/Lab-2/README.md`, step by step.

Today



The steps are those in the README. Start with the **smoke test** (step 2): it checks the scripts without a GPU. If it has not passed by minute 15, put your hand up.

The full experiment is **one GPU job**. It scores the prompts, trains the model and runs the general-knowledge check, and it can take an hour or more. Submit it by minute 60, even with a first version of your prompt.

GPUs are reserved until about 20:00, so jobs can finish after the session.

Your data and your prompt

```
# data/my-task/train.jsonl: one example per line
{"id": "train-00001", "text": "I left before the end.", "label": "negative"}

# lab2lib/prompting.py: the two settings you change
TASK_FIELDS = ("text",)
TASK_SYSTEM = ("Is the review positive or negative? "
               "Use exactly one of: positive, negative. "
               "End with 'Label: <label>'.")
```

Three splits of your data

- **train**: what the model learns from (1,000–5,000)
- **dev**: for choosing the prompt and settings
- **test**: only for the final scores

`make_splits.py` creates the three files from your data, and `check_data.py` checks them.

Keep the last sentence of the instruction: the scorer looks for `Label: <label>`. Read your prompts in a test run without a GPU before you submit the real job.

What the script compares

	Zero-shot instructions only	Few-shot + 5 examples
Base model		
Instruct model		

A **base model** has only learned to continue text; an **instruct model** has also learned to follow instructions. The best of the four prompts on the dev set is *the* baseline.

Fine-tuned model. The instruct model trained on your data with LoRA, which trains a small add-on (an *adapter*) instead of the whole model.

General-knowledge check. 50 questions unrelated to your task, asked before and after training. A large drop means training damaged the model. A change of a few points can be chance.

Read the outputs before you trust a score

A score of exactly 0 or 100 usually means the scorer is wrong, not the model. Read ten raw outputs with `show_examples.py --run <name>`.

If you have time: tune the training

Setting	Start	What it controls
learning rate	0.0002	how large each training step is
epochs	2	how many times the model sees the training data
LoRA rank	16	the size of the adapter

Change *one* setting at a time, give each run its own name, and keep a small table of the runs.

Lower training loss is not a result

The **loss** measures how wrong the model's answers are. If the training loss falls while the dev loss rises, the model is **memorising** the training examples.

Then try fewer epochs or a lower learning rate. Choose settings on the dev loss, never on the test score.

Checklist for today

- [ ] smoke test passed (step 2)
- [ ] example job finished on `gputest` (step 4)
- [ ] splits made and `check_data.py` clean (step 5)
- [ ] prompt adapted and read in a test run (steps 6–7)
- [ ] full job submitted to `gpumedium` (step 8)
- [ ] table printed with `summarize_results.py`
- [ ] ten raw outputs read with `show_examples.py`
- [ ] results folder and config kept for the report

Good practice on Roihu

- follow a job with `squeue --me` and `tail -f logs/lab2-lora-<jobid>.out`
- do not run models on the login node
- close GPU Jupyter sessions you are not using

Stuck for more than ten minutes? Put your hand up. The README has a troubleshooting table.